FlashNorm: Fast and Exact Normalization for Transformers

Nils Graef[*], **Filip Makraduli**, **Andrew Wasielewski**, **Matthew Clapp**
OpenMachine

Abstract

Normalization layers are ubiquitous in large language models (LLMs) yet represent a compute bottleneck: on hardware with distinct vector and matrix execution units, the RMS calculation blocks the subsequent matrix multiplication, preventing parallel execution. We present **FlashNorm**, an *exact* reformulation of RMSNorm followed by a linear layer that (i) eliminates the normalization weight tensor by folding it into the subsequent linear layer, and (ii) defers the scalar RMS normalization to the output of the matrix multiplication, enabling the two operations to execute in parallel. FlashNorm is mathematically identical to the original computation—it introduces no approximation and requires no retraining. The same technique extends to LayerNorm, Dynamic Tanh (DyT),feed-forward networks with GLU variants, RoPE-based attention, and QK-normalization. On an NVIDIA T4 GPU, FlashNorm achieves **33–36%** lower latency on the norm-plus-projection operation in the compute-bound (prefill) regime, and up to **14%** lower end-to-end latency at Llama-7B scale with 4096 tokens. We verify zero-loss weight folding on SmolLM2-135M (max logit difference $= 0$, cosine similarity $= 1.0$). Beyond inference speed, FlashNorm simplifies model implementations by reducing parameter tensor count, analogous to the simplification achieved by PaLM's removal of bias-parameters from all linear layers. Watch our explainer video [1] and see [2, 3, 4, 5] for code and more transformer tricks.

1 Introduction

Normalization is indispensable to the training stability and generalization of modern LLMs. RMSNorm [6], used by Llama [7], Gemma [8], Mistral [9], and OLMo 2 [10], normalizes each activation vector by its root mean square before scaling by learned weights. In the standard transformer network [11], RMSNorm is followed immediately by a linear (projection) layer—in both the attention and feed-forward sub-networks of every transformer block.

This sequential structure conceals a hardware inefficiency. On processors with dedicated vector units (for element-wise ops) and matrix units (for GEMMs), the RMS value must be fully computed *before* the linear layer can begin, because the normalized activations are its input. The matrix unit therefore sits idle during the RMS calculation—a bottleneck that compounds across all transformer layers and inference calls.

We address this bottleneck with **FlashNorm**, which makes two mathematically equivalent transformations to the RMSNorm-then-linear pipeline shown in Fig. 1(a):

1. **Weightless normalization.** The per-channel normalization weights g_i are absorbed into the weight matrix of the following linear layer, eliminating them as a separate parameter tensor as illustrated in Fig. 1(b).

[*] info@openmachine.ai

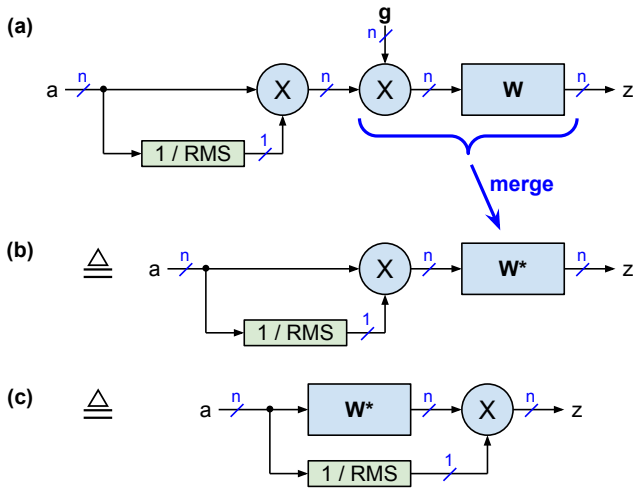

Figure 1: Mathematically identical implementations of RMSNorm followed by a linear layer. (a) Standard: normalization weights $\mathbf{g}$ are applied element-wise before $\mathbf{W}$. (b) Weightless: $\mathbf{g}$ is folded into $\mathbf{W}^* = \mathrm{diag}(\mathbf{g})\mathbf{W}$. (c) **FlashNorm**: normalization is also deferred to the output, enabling the GEMM ($\mathbf{W}^*$) and the RMS calculation to execute in parallel. $\triangleq$ denotes mathematical identity.

2. **Deferred normalization.** Because scaling by $1/\mathrm{RMS}(\mathbf{a})$ commutes with matrix multiplication, the scalar normalization is moved to the *output* of the linear layer as shown in Fig. 1(c). This allows the GEMM and the RMS calculation to execute in parallel on separate hardware units.

The name is inspired by FlashAttention [12], where the softmax denominator is similarly deferred past a matrix multiplication to enable parallel processing of keys and values.

FlashNorm requires no retraining, and weight folding can be applied post-hoc to any pretrained checkpoint. We further show how the same ideas extend to Layer Normalization [13], DyT [14], GLU-variant FFNs [15], RoPE-based attention [16], and QK-normalization [17].

Contributions.

- We formally characterize the commutativity of RMS normalization with bias-free matrix multiplication (Proposition 2) and derive the exact conditions under which deferred normalization holds.
- We present FlashNorm as a unified framework covering RMSNorm, LayerNorm, DyT, GLU-FFNs, RoPE attention, and QK-norm.
- We provide a complete hardware timing analysis demonstrating the bottleneck that FlashNorm eliminates.
- We report controlled GPU benchmarks at two model scales and provide a reproducible Colab notebook alongside zero-loss weight-folding verification.

2 Background and Related Work

Normalization in transformers. Layer Normalization [13] applies mean centering followed by variance normalization with learned scale and bias. RMSNorm [6] drops mean centering and bias, reducing to pure RMS-based scaling; it has become the dominant normalization choice in recent open LLMs due to its simplicity and comparable empirical performance. Dynamic Tanh (DyT) [14] is a recently proposed drop-in replacement that eliminates the normalization computation entirely, replacing it with a parameterized $\tanh$; FlashNorm is complementary in that it targets the interaction between normalization and the subsequent linear layer.

Kernel fusion and operation reordering. Fusing small operations into a single GPU kernel is a standard optimization to reduce memory bandwidth and kernel-launch overhead. Mehta et al. [18]

report that modifications to the layer normalization implementation significantly affect tokens-per-second throughput, attributing this to a lack of kernel fusion. FlashNorm reduces the number of kernel launches by eliminating the normalization weight application as a separate step, and additionally enables parallelism between the vector and matrix units that kernel fusion alone cannot achieve.

Weight folding and parameter elimination. The idea of absorbing one layer's parameters into a neighboring layer appears in several forms. Batch normalization folding into convolutional weights is standard practice in quantization-aware training and model deployment [19, 20]. PaLM [21] removes bias parameters from all linear layers, simplifying the computation graph. FlashNorm applies analogous reasoning to normalization weights, showing that they are redundant given access to the subsequent linear layer's weights.

FlashAttention. FlashAttention [12] is the closest conceptual predecessor: it defers the softmax normalization (division by $\sum \exp$) past the multiplication with the value matrix V, enabling online computation and improving memory efficiency. FlashNorm applies the same reordering insight to the simpler setting of RMS normalization and a dense linear layer.

3 FlashNorm

We now develop FlashNorm formally. All propositions assume bias-free linear layers unless stated otherwise; Section 3.3 addresses the biased case.

3.1 Weightless Normalization

Let $\mathbf{a} \in \mathbb{R}^n$ be an activation vector. RMSNorm computes

$$\mathrm{RMSNorm}(\mathbf{a})_i = \frac{a_i}{\mathrm{RMS}(\mathbf{a})} \cdot g_i, \qquad \mathrm{RMS}(\mathbf{a}) = \sqrt{\frac{1}{n}\sum_{i=1}^{n} a_i^2},$$

where $\mathbf{g} \in \mathbb{R}^n$ are learned normalization weights. In transformer models, RMSNorm is followed by a linear layer $\mathbf{W} \in \mathbb{R}^{n\times k}$ as illustrated in Fig. 1(a):

$$\mathbf{z} = \mathrm{RMSNorm}(\mathbf{a})\,\mathbf{W}.$$

Proposition 1 (Weightless normalization)**.** *Let $\mathbf{W}^* \in \mathbb{R}^{n\times k}$ be defined by $W^*_{i,j} = g_i \cdot W_{i,j}$. Then $\mathbf{z} = \frac{\mathbf{a}}{\mathrm{RMS}(\mathbf{a})}\mathbf{W}^*$, i.e., the normalization weights $\mathbf{g}$ can be absorbed into $\mathbf{W}$ and need not be stored or applied separately.*

Proof. $z_j = \sum_i \frac{a_i}{\mathrm{RMS}(\mathbf{a})} g_i W_{i,j} = \sum_i \frac{a_i}{\mathrm{RMS}(\mathbf{a})} W^*_{i,j}$. □

This proposition holds for linear layers with or without a bias term at the output. The practical implication is that the normalization weight vector $\mathbf{g}$ becomes a redundant parameter: it can be folded into $\mathbf{W}$ once at load time, saving memory, reducing the parameter count, and eliminating one vector-multiply kernel call per forward pass.

3.2 Deferred Normalization

Proposition 2 (Commutativity of RMS scaling with bias-free GEMM)**.** *Let $\mathbf{W}^*$ be as in Proposition 1 and let the linear layer be bias-free. Then*

$$\mathbf{z} = \frac{\mathbf{a}}{\mathrm{RMS}(\mathbf{a})}\mathbf{W}^* = (\mathbf{a}\,\mathbf{W}^*) \cdot \frac{1}{\mathrm{RMS}(\mathbf{a})}.$$

That is, the scalar normalization $1/\mathrm{RMS}(\mathbf{a})$ can be applied after *the matrix multiplication.*

Proof. Scalar multiplication commutes with matrix multiplication: $(\alpha\mathbf{a})\mathbf{W}^* = (\mathbf{a}\mathbf{W}^*)\alpha$ for any scalar α. Setting $\alpha = 1/\mathrm{RMS}(\mathbf{a})$ gives the result. □

Remark 1 (Bias case)**.** *If the linear layer has a bias* $\mathbf{c} \in \mathbb{R}^k$*, deferred normalization does not hold directly because* $(\mathbf{a}\mathbf{W}^* + \mathbf{c})/\mathrm{RMS}(\mathbf{a}) \neq \mathbf{a}\mathbf{W}^*/\mathrm{RMS}(\mathbf{a}) + \mathbf{c}$*. In this case, Proposition 1 (weightless normalization) still applies, but the normalization must precede the bias addition.*

Corollary 1 (Parallel execution)**.** *Under the conditions of Proposition 2, computing* $\mathbf{a}\mathbf{W}^*$ *and* $\mathrm{RMS}(\mathbf{a})$ *are* independent *computations that may proceed in parallel. The final result requires only a single vector-scalar multiply after both complete.*

Figure 1 illustrates the three mathematically equivalent implementations: (a) standard RMSNorm-then-linear, (b) weightless normalization, and (c) the full FlashNorm with deferred normalization.

3.3 Support for Normalization Bias

LayerNorm [13] and DyT [14] include an additive bias $\boldsymbol{\beta}$ after scaling by $\mathbf{g}$. Fig. 2 illustrates how this bias can be moved to the output of the following linear layer, resulting in a modified bias $\mathbf{c}^* = \mathbf{c} + \boldsymbol{\beta}\mathbf{W}$, see Fig. 2(b). After this elimination of $\boldsymbol{\beta}$, the normalization weights $\mathbf{g}$ can be folded into the linear layer as per Proposition 1. Note that deferred normalization (Proposition 2) does not apply once a bias is present; the scalar $1/\mathrm{RMS}(\mathbf{a})$ must be applied before adding $\mathbf{c}^*$.

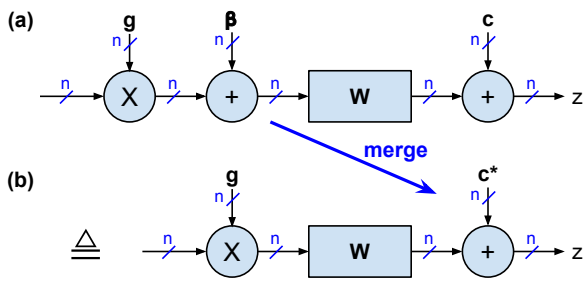

Figure 2: Elimination of bias $\boldsymbol{\beta}$: (a) Before elimination with $\boldsymbol{\beta}$ between normalization weights $\mathbf{g}$ and linear layer. (b) Optimized version with new bias term $\mathbf{c}^* = \mathbf{c} + \boldsymbol{\beta}\mathbf{W}$ at the output.

3.4 Folding Mean Centering into a Preceding Linear Layer

LayerNorm applies mean centering before RMSNorm. If mean centering is preceded by a linear layer with weights $\mathbf{V} \in \mathbb{R}^{n \times n}$, the centering can be absorbed into $\mathbf{V}$ without retraining. Fig. 3(a) shows the weight matrix $\mathbf{V}$ followed by mean centering and RMSNorm.

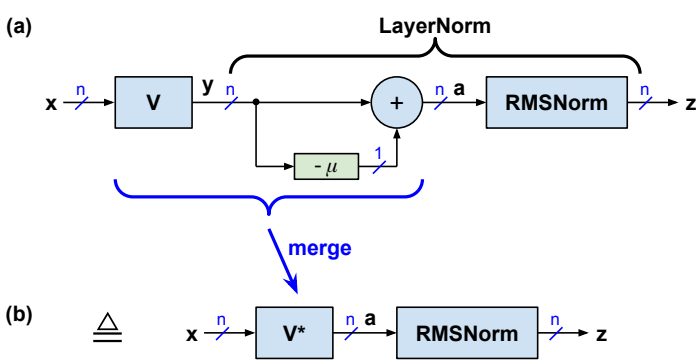

Figure 3: Elimination of mean centering: (a) Original weight matrix $\mathbf{V}$ followed by mean centering. (b) Optimized version where the mean centering is merged into the modified weight matrix $\mathbf{V}^*$.

The mean μ is calculated from the linear layer outputs y_j as $\mu = \frac{1}{n}\sum_{j=1}^{n} y_j$ where $y_j = \sum_{i=1}^{n} x_i V_{i,j}$. Let $s_i = \sum_j V_{i,j}$ be the row sum of row i, then we can calculate μ directly from the inputs x_i as

$$\mu = \frac{1}{n}\sum_{j=1}^{n}\sum_{i=1}^{n} x_i V_{i,j} = \frac{1}{n}\sum_{i=1}^{n} x_i \left[\sum_{j=1}^{n} V_{i,j}\right] = \frac{1}{n}\sum_{i=1}^{n} x_i s_i.$$

The mean-centered outputs are

$$a_j = y_j - \mu = \sum_{i=1}^{n} x_i V_{i,j} - \frac{1}{n}\sum_{i=1}^{n} x_i s_i = \sum_{i=1}^{n} x_i \underbrace{\left(V_{i,j} - \frac{s_i}{n}\right)}_{V^*_{i,j}},$$

so the modified weights $V^*_{i,j} = V_{i,j} - s_i/n$ absorb the mean centering entirely, see Fig. 3(b). This also provides a lossless recipe for converting a LayerNorm model to RMSNorm without any retraining.

4 Extensions

4.1 FlashNorm for FFNs with ReLU

For bias-free FFNs with ReLU, the scale-invariance $\mathrm{ReLU}(s \cdot \mathbf{a}) = s \cdot \mathrm{ReLU}(\mathbf{a})$ for $s \geq 0$ [22] allows the RMS normalization to be deferred all the way to the FFN output, past both the up-projection and the ReLU nonlinearity as illustrated in Fig. 4(b). If the up-projection expands from n to f channels, this saves $f - n$ multiply operations compared to applying the scaling at the FFN input.

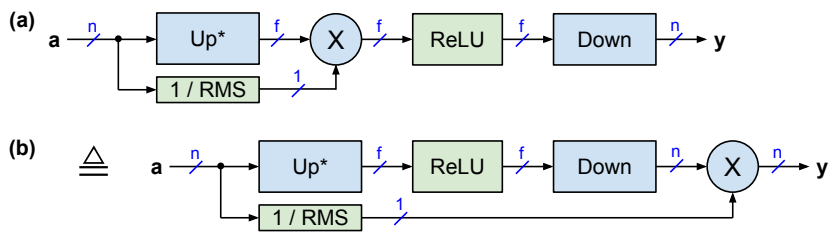

Figure 4: FFN with ReLU and preceding flash normalization: (a) unoptimized version; (b) optimized version where the normalization is deferred to the output of the FFN. Up and Down denote the linear layers for up and down projections.

4.2 FlashNorm for FFNs with GLU Variants

For GLU-variant FFNs [15], the FlashNorm at the FFN input requires scaling at both the Gate and Up projection outputs, see Fig. 5(a). One of these scaling operations can be deferred to the FFN output, saving $f - n$ multiply operations as shown in Fig. 5(b).

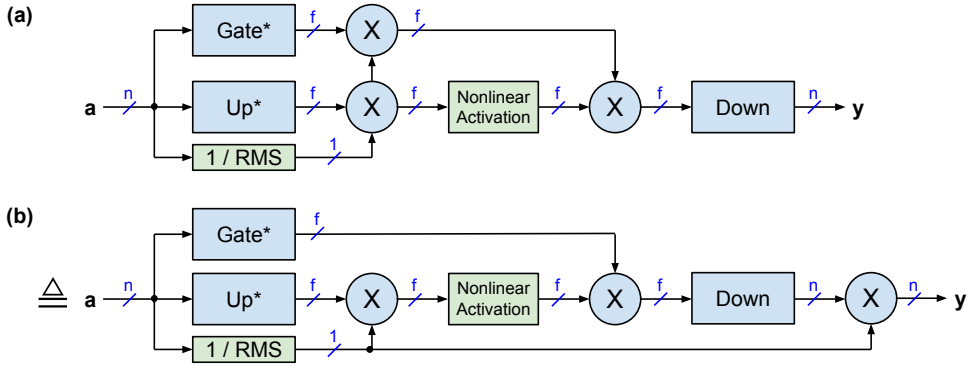

Figure 5: FFN with GLU variant and preceding flash normalization: (a) unoptimized version; (b) optimized version with fewer scaling multipliers.

Special case: ReGLU and Bilinear GLU. When the activation function is ReLU (ReGLU) or linear (bilinear GLU) [15], both scaling operations can be consolidated at the output as illustrated in Fig. 6, using the reciprocal of the *squared* RMS (equivalently, the reciprocal of the mean square $\mathrm{MS}(\mathbf{a})$):

$$\frac{1}{(\mathrm{RMS}(\mathbf{a}))^2} = \frac{1}{\mathrm{MS}(\mathbf{a})} = \frac{n}{\sum_{i=1}^{n} a_i^2}.$$

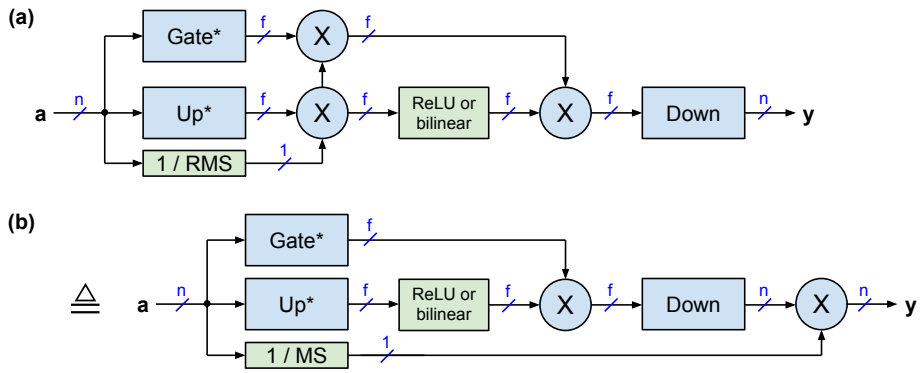

Figure 6: FFN with ReGLU (or bilinear GLU) and preceding flash normalization: (a) unoptimized version; (b) optimized version, which saves $2f - n$ multiply operations.

4.3 FlashNorm for Attention with RoPE

For the query ($\mathbf{Q}^*$) and key ($\mathbf{K}^*$) projections in RoPE-based attention [16], the $1/\mathrm{RMS}(\mathbf{a})$ factor can be fused with the precomputed RoPE cosine and sine tables, which are shared across all attention heads as shown in Fig. 7. Specifically, replacing $\cos(\cdot)$ with $\cos(\cdot)/\mathrm{RMS}(\mathbf{a})$ (and similarly for $\sin$) eliminates the per-head scaling entirely, saving $2hH - h$ multiply operations, where h is the head dimension and H is the number of heads. Additionally, the $1/\sqrt{h}$ scaled dot-product factor can be folded into the same scaling operation.

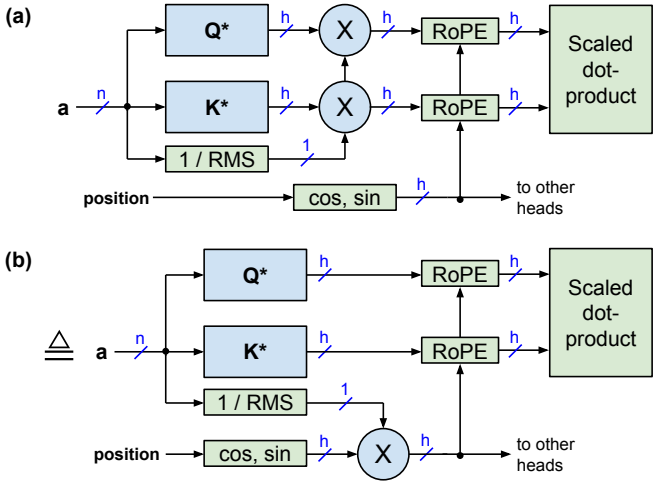

Figure 7: Flash normalization for scaled dot-product attention with RoPE: (a) unoptimized version; (b) optimized version where the normalization is fused with $\cos(\cdot)$ and $\sin(\cdot)$.

4.4 QK-Normalization

Gemma 3 [23], OLMo 2 [10], OpenELM [18] and other LLMs use per-head QK-normalization [17], which applies RMSNorm to queries and keys *after* the projection, preventing direct application of FlashNorm. Two optimizations are still available:

Eliminating the pre-projection RMS. By the scale-invariance of RMS, $\mathrm{RMS}(s \cdot \mathbf{a}) = s \cdot \mathrm{RMS}(\mathbf{a})$. When a FlashNorm RMS scaler s_a is followed immediately by a QK-norm RMS scaler s_c on the projected output as shown in Fig. 8 with activation vectors $\mathbf{a}, \mathbf{b}, \mathbf{c}$, the two scalers cancel:

$s_a \cdot s_c = s_a \cdot (s_b/s_a) = s_b$ because $s_c = 1/\text{RMS}(\mathbf{c}) = 1/\text{RMS}(\mathbf{b} \cdot s_a) = 1/\left(s_a \cdot \text{RMS}(\mathbf{b})\right) = s_b/s_a$. The pre-projection RMS calculation is therefore redundant and can be eliminated as illustrated in Fig. 8(b). The scale-invariance property of $\text{RMS}(\vec{a})$ doesn't hold exactly true for RMS with epsilon (see appendix). This should not matter because the epsilon only makes an impact if the RMS (or energy) of the activation vector is very small, in which case the epsilon limits the up-scaling of this low-energy activation vector.

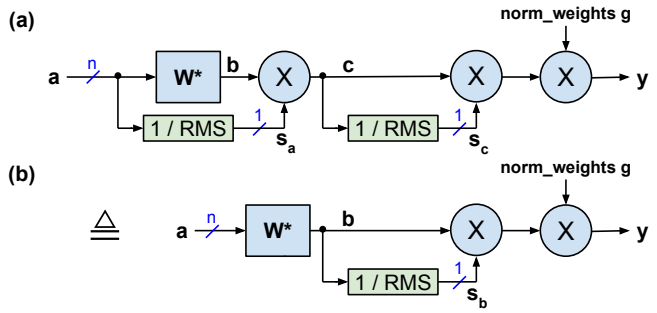

Figure 8: Linear layer with flash normalization followed by a second normalization: (a) unoptimized version; (b) optimized version.

Fusing QK-norm weights with RoPE. If QK-norm weights are shared across all heads of a layer (as in Gemma 3 and OpenELM), they can be fused with the RoPE cosine and sine vectors, which are already shared across heads, see Fig. 9. The fused tables are computed once per layer, and the normalization weights require only a permutation to align with RoPE's interleaved layout: $\text{permute}(\mathbf{g}) = (g_2, g_1, g_4, g_3, \ldots, g_h, g_{h-1})$.

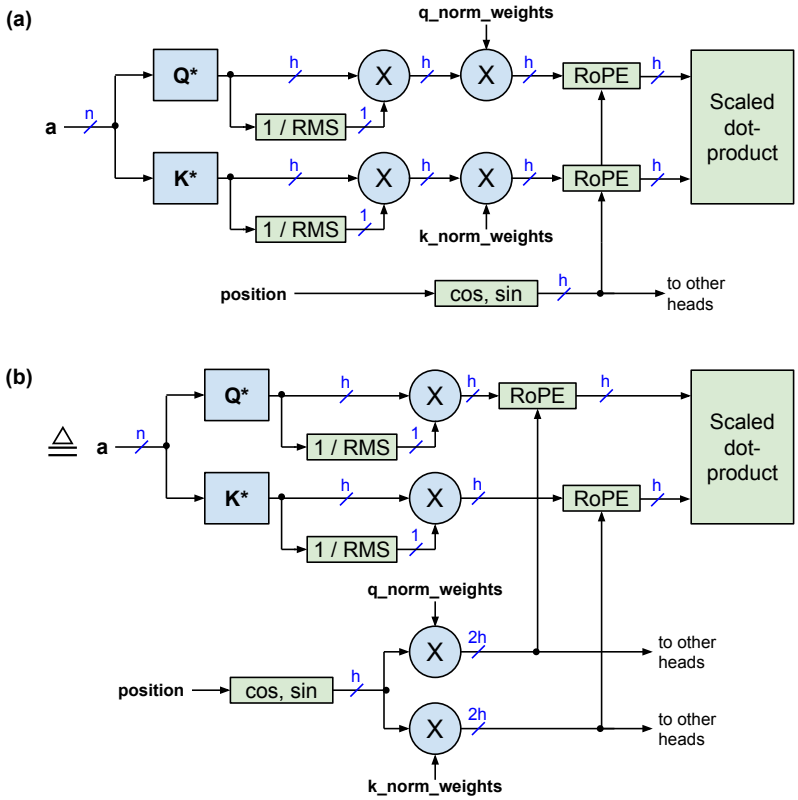

Figure 9: QK-norm with RoPE: (a) unoptimized version; (b) optimized version.

5 Hardware Analysis

We analyze the compute bottleneck that FlashNorm eliminates. Consider a processor with one vector unit (throughput: m ops/cycle) and one matrix unit (throughput: m^2 ops/cycle) where m is the processor width. Specifically:

- Multiplying an n-vector with an $n \times n$ matrix takes n^2 MAD (multiply-add) operations, which takes n^2/m^2 matrix cycles.
- Computing $1/\mathrm{RMS}(\mathbf{a})$ requires n MAD operations (for squaring and adding) plus 2 scalar ops (for $\sqrt{n/x}$), which takes n/m vector cycles if we ignore the 2 scalar ops.
- Scaling an n-vector by a scalar takes n/m vector cycles.

Fig. 10 shows timing diagrams for the example $n = 512$ and $m = 128$. Without FlashNorm (sequential execution), the matrix unit cannot begin until both the RMS and the scaling are complete, imposing an 8 cycle bubble as shown in Fig. 10(a). The subsequent vector-matrix multiply takes $(512)^2/(128)^2 = 16$ cycles; the total is 24 cycles.

With row-major ordering, the first rows of $\mathbf{W}^*$ can be processed while later rows of the RMS accumulation are still in flight, reducing the bubble to 5 cycles (total: 21 cycles), but the bottleneck persists, see Fig. 10(b).

With FlashNorm (deferred normalization), the vector-matrix multiply ($\mathbf{a}\mathbf{W}^*$) and the RMS calculation proceed fully in parallel on separate units as shown in Fig. 10(c). The scaling at the end can be performed in parallel to the matrix unit if $\mathbf{W}^*$ is processed in column-major order. Total execution: 17 cycles—a **29% reduction** versus the sequential baseline.

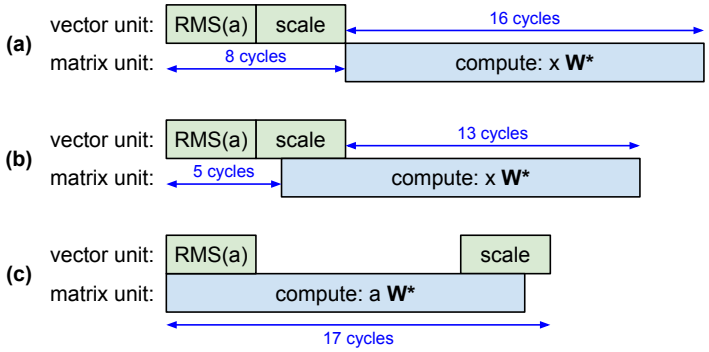

Figure 10: Timing diagrams for $n = 512, m = 128$. (a) Sequential: matrix unit idles 8 cycles. (b) Partial overlap with interleaved scaling and vector-matrix multiplication. (c) **FlashNorm**: vector-matrix multiply and RMS run fully in parallel; 17 cycles total, a 29% reduction over (a).

6 Experiments and conclusions

Refer to [24, 3] for Python code that demonstrates the mathematical equivalency of the optimizations presented in this paper. A reproducible Colab notebook for the GPU benchmarks below is available in the `transformer-tricks` repository[2].

We verified lossless weight folding on SmolLM2-135M [25] using `flashify_repo()` from the `transformer-tricks` package [3]. The original and weight-folded models produce identical logits (max difference = 0.0, cosine similarity = 1.0) and identical generated text.

To validate the parallel execution described in Fig. 10, we benchmarked the norm-then-project operation on an NVIDIA T4 GPU (65 TFLOPS FP16, 320 GB/s). We compare sequential execution (RMSNorm $\rightarrow$ GEMM) against FlashNorm with adaptive dispatch: when the GEMM is compute-bound (many tokens), the GEMM runs on tensor cores and $1/\mathrm{RMS}(\vec{a})$ runs on vector cores in parallel

[2] https://github.com/OpenMachine-ai/transformer-tricks/blob/main/notebooks/flashNorm_gpu_benchmark.ipynb

via separate CUDA streams; when memory-bound (few tokens), FlashNorm falls back to sequential execution to avoid stream overhead. Table 1 shows the results at SmolLM2-135M scale, and Table 2 at Llama-7B scale.

Table 1: Norm+project latency on T4 GPU at SmolLM2-135M scale (n=576, k=960). Below the compute-bound threshold, FlashNorm falls back to sequential execution (0% overhead).

Tokens	Sequential	FlashNorm	Speedup	Path
1	0.374 ms	0.374 ms	0.0%	sequential (fallback)
64	0.506 ms	0.506 ms	0.0%	sequential (fallback)
4096	0.704 ms	0.468 ms	+33.6%	parallel (overlap)
8192	0.929 ms	0.599 ms	+35.5%	parallel (overlap)

Table 2: Norm+project latency on T4 GPU at Llama-7B scale (n=4096, k=4096).

Tokens	Sequential	FlashNorm	Speedup	Path
1	0.641 ms	0.641 ms	0.0%	sequential (fallback)
64	0.307 ms	0.307 ms	0.0%	sequential (fallback)
256	0.577 ms	0.577 ms	0.0%	sequential (fallback)
1024	1.882 ms	1.654 ms	+12.1%	parallel (overlap)
4096	7.628 ms	6.570 ms	+13.9%	parallel (overlap)

FlashNorm uses adaptive dispatch: the parallel path is used only when the GEMM is compute-bound (token count exceeds a hardware-dependent threshold). For memory-bound workloads such as autoregressive decoding, FlashNorm falls back to sequential execution (0% overhead). This is because managing parallel CUDA streams introduces scheduling overhead that exceeds the time saved when the GEMM is small. Profiling individual operations within a SmolLM2-135M decoder layer at 4096 tokens confirms that the $1/\text{RMS}(\vec{a})$ calculation takes $\sim$0.08 ms per layer. Across 30 layers, the theoretical maximum savings from perfect overlap is $\sim$5 ms out of $\sim$60 ms total ($\sim$8%).

While the speedup on the norm+project operation itself is significant (33–35%), the end-to-end model speedup is modest because this operation is a fraction of total layer time. For comparison, we measured a throughput of 204 tokens per second for OpenELM-270M with 4-bit weight quantization using the MLX framework on an M1 MacBook Air. This throughput increases to only 225 tokens per second when we remove RMSNorm entirely, giving an upper bound of $\leq$ 10% for this model. Native integration into inference frameworks at the kernel level—where stream management overhead is negligible—would allow the 33–35% operation-level improvement to translate more directly into end-to-end gains, particularly for larger models with bigger GEMMs.

For many applications, the main advantage of FlashNorm is simplification. This is similar to the simplifications we get from using RMSNorm over Layer Normalization (LayerNorm [13]), and from PaLM's removal of bias-parameters from all linear layers [21].

Future work includes integrating FlashNorm into popular frameworks such as HuggingFace Transformers [26], whisper.cpp [27], llama.cpp [28], vLLM [29], llamafile [30], LM Studio [31], Ollama [32], SGLang [33], and combining it with parameter quantization.

Acknowledgments

We would like to thank Dmitry Belenko for helpful feedback on this work.

A RMS with epsilon

Many implementations add a small epsilon ϵ to the RMS value to limit the resulting scaling factor $1/\text{RMS}(\vec{a})$ and to avoid division by zero as follows:

$$\text{RMSe}(\vec{a}) = \sqrt{\epsilon + \frac{1}{n}\sum_{i=1}^{n} a_i^2} = \sqrt{\epsilon + (\text{RMS}(\vec{a}))^2}$$

RMSe($\vec{a}$) can be used as a drop-in-replacement for RMS. The popular HuggingFace transformer library calls this epsilon `rms_norm_eps`, which is set to 10^{-5} for Llama3.

B Eliminating $1/n$

This section details a small optimization that eliminates the constant term $1/n$ from the RMS calculation. First, we factor out $1/n$ as follows:

$$\text{RMS}(\vec{a}) = \sqrt{\frac{1}{n}\sum_{i=1}^{n} a_i^2} = \sqrt{\frac{1}{n}}\sqrt{\sum_{i=1}^{n} a_i^2} = \sqrt{\frac{1}{n}} \cdot \text{RSS}(\vec{a})$$

where $\text{RSS}(\vec{a}) = \sqrt{\sum_{i=1}^{n} a_i^2}$. We can now merge the constant term into the normalization weights g_i as follows:

$$y_i = \frac{a_i}{\text{RMS}(\vec{a})} \cdot g_i = \frac{a_i}{\text{RSS}(\vec{a})}\sqrt{n} \cdot g_i = \frac{a_i}{\text{RSS}(\vec{a})} \cdot g_i^*$$

with new normalization weights $g_i^* = \sqrt{n} \cdot g_i$. These new normalization weights can now be merged with the weights $\mathbf{W}$ of the following linear layer as shown in the previous sections. This optimization also applies for the case where we add an epsilon as detailed in the previous section. In this case, we factor out $1/n$ as follows:

$$\text{RMSe}(\vec{a}) = \sqrt{\epsilon + \frac{1}{n}\sum_{i=1}^{n} a_i^2} = \sqrt{\frac{1}{n}\left(n\epsilon + \sum_{i=1}^{n} a_i^2\right)} = \sqrt{\frac{1}{n}} \cdot \text{RSSe}(\vec{a})$$

where $\text{RSSe}(\vec{a}) = \sqrt{n\epsilon + \sum_{i=1}^{n} a_i^2}$.

References

[1] Christopher Marquand with audio generated by Notebook LM. Podcast-video about FlashNorm. May 2025.

[2] Nils Graef and Andrew Wasielewski. Slim attention: cut your context memory in half without loss – K-cache is all you need for MHA. March 2025. *arXiv:2503.05840*.

[3] OpenMachine. Transformer tricks. 2024. URL `https://github.com/OpenMachine-ai/transformer-tricks`.

[4] Nils Graef. KV-weights are all you need for skipless transformers. April 2024. *arXiv:2404.12362*.

[5] Nils Graef. Transformer tricks: Precomputing the first layer. February 2024. *arXiv:2402.13388*.

[6] Biao Zhang and Rico Sennrich. Root mean square layer normalization. October 2019. *arXiv:1910.07467*.

[7] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, Aurelien Rodriguez, Armand Joulin, Edouard Grave, and Guillaume Lample. LLaMA: Open and efficient foundation language models. February 2023. *arXiv:2302.13971*.

[8] Gemma Team, Google DeepMind. Gemma: Open Models Based on Gemini Research and Technology. 2024.

[9] Albert Q Jiang, Alexandre Sablayrolles, Arthur Mensch, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Florian Bressand, Gianna Lengyel, Guillaume Lample, Lucile Saulnier, Lélio Renard Lavaud, Marie-Anne Lachaux, Pierre Stock, Teven Le Scao, Thibaut Lavril, Thomas Wang, Timothée Lacroix, and William El Sayed. Mistral 7B. October 2023. *arXiv:2310.06825*.

[10] Team OLMo, Pete Walsh, Luca Soldaini, Dirk Groeneveld, Kyle Lo, Shane Arora, Akshita Bhagia, Yuling Gu, Shengyi Huang, Matt Jordan, Nathan Lambert, Dustin Schwenk, Oyvind Tafjord, Taira Anderson, David Atkinson, Faeze Brahman, Christopher Clark, Pradeep Dasigi, Nouha Dziri, Michal Guerquin, Hamish Ivison, Pang Wei Koh, Jiacheng Liu, Saumya Malik,

William Merrill, Lester James V Miranda, Jacob Morrison, Tyler Murray, Crystal Nam, Valentina Pyatkin, Aman Rangapur, Michael Schmitz, Sam Skjonsberg, David Wadden, Christopher Wilhelm, Michael Wilson, Luke Zettlemoyer, Ali Farhadi, Noah A Smith, and Hannaneh Hajishirzi. 2 OLMo 2 Furious. December 2024. *arXiv:2501.00656*.

[11] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. June 2017. *arXiv:1706.03762*.

[12] Tri Dao, Daniel Y Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. FlashAttention: Fast and memory-efficient exact attention with IO-awareness. May 2022. *arXiv:2205.14135*.

[13] Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E Hinton. Layer Normalization. July 2016. *arXiv:1607.06450*.

[14] Jiachen Zhu, Xinlei Chen, Kaiming He, Yann LeCun, and Zhuang Liu. Transformers without Normalization. 2025. *arXiv:2503.10622*.

[15] Noam Shazeer. GLU Variants Improve Transformer. February 2020. *arXiv:2002.05202*.

[16] Jianlin Su, Yu Lu, Shengfeng Pan, Ahmed Murtadha, Bo Wen, and Yunfeng Liu. RoFormer: Enhanced transformer with Rotary Position Embedding. April 2021. *arXiv:2104.09864*.

[17] Alex Henry, Prudhvi Raj Dachapally, Shubham Pawar, and Yuxuan Chen. Query-key normalization for transformers. October 2020. *arXiv:2010.04245*.

[18] Sachin Mehta, Mohammad Hossein Sekhavat, Qingqing Cao, Maxwell Horton, Yanzi Jin, Chenfan Sun, Iman Mirzadeh, Mahyar Najibi, Dmitry Belenko, Peter Zatloukal, and Mohammad Rastegari. OpenELM: An efficient language model family with open-source training and inference framework. April 2024. *arXiv:2404.14619*.

[19] Benoit Jacob, Skirmantas Kligys, Bo Chen, Menglong Zhu, Matthew Tang, Andrew Howard, Hartwig Adam, and Dmitry Kalenichenko. Quantization and training of neural networks for efficient integer-arithmetic-only inference. December 2017. *arXiv:1712.05877*.

[20] Kaichao You, Guo Qin, Anchang Bao, Meng Cao, Ping Huang, Jiulong Shan, and Mingsheng Long. Efficient ConvBN blocks for transfer learning and beyond. May 2023. *arXiv:2305.11624*.

[21] Aakanksha Chowdhery, Sharan Narang, Jacob Devlin, Maarten Bosma, Gaurav Mishra, Adam Roberts, Paul Barham, Hyung Won Chung, Charles Sutton, Sebastian Gehrmann, Parker Schuh, Kensen Shi, Sasha Tsvyashchenko, Joshua Maynez, Abhishek Rao, Parker Barnes, Yi Tay, Noam Shazeer, et al. PaLM: Scaling language modeling with Pathways. April 2022. *arXiv:2204.02311*.

[22] Wikipedia. Rectifier (neural networks), 2024. Accessed June-2024.

[23] Gemma Team, Aishwarya Kamath, Johan Ferret, Shreya Pathak, Nino Vieillard, Ramona Merhej, Sarah Perrin, Tatiana Matejovicova, Alexandre Ramé, Morgane Rivière, Louis Rouillard, Thomas Mesnard, Geoffrey Cideron, Jean-Bastien Grill, Sabela Ramos, Edouard Yvinec, Michelle Casbon, Etienne Pot, Ivo Penchev, Gaël Liu, Francesco Visin, Kathleen Kenealy, Lucas Beyer, Xiaohai Zhai, Anton Tsitsulin, Robert Busa-Fekete, Alex Feng, Noveen Sachdeva, Benjamin Coleman, Yi Gao, Basil Mustafa, Iain Barr, Emilio Parisotto, David Tian, Matan Eyal, Colin Cherry, Jan-Thorsten Peter, Danila Sinopalnikov, et al. Gemma 3 Technical Report. March 2025. *arXiv:2503.19786*.

[24] OpenMachine. FlashNorm. 2024. URL https://huggingface.co/open-machine/FlashNorm.

[25] Loubna Ben Allal, Anton Lozhkov, Elie Bakouch, Gabriel Martín Blázquez, Lewis Tunstall, Agustín Piqueres, Andres Marafioti, Cyril Zakka, Leandro von Werra, and Thomas Wolf. SmolLM2 - with great data, comes great performance, 2024. URL https://huggingface.co/HuggingFaceTB/SmolLM2-1.7B.

[26] HuggingFace. Transformers. URL https://huggingface.co/docs/transformers.

[27] Georgi Gerganov. whisper.cpp. . URL https://github.com/ggml-org/whisper.cpp.

[28] Georgi Gerganov. llama.cpp. . URL https://github.com/ggml-org/llama.cpp.

[29] vLLM Project. vLLM. URL https://github.com/vllm-project/vllm.

[30] Mozilla. llamafile. URL https://github.com/Mozilla-Ocho/llamafile.

[31] LM Studio. LM Studio. URL https://lmstudio.ai.

[32] Ollama. Ollama. URL https://github.com/ollama/ollama.

[33] SGLang. SGLang. URL https://github.com/sgl-project/sglang.